Barclays Full Stack Engineer - What to Learn

Why these keywords

This is a Java/Spring Boot backend role paired with a modern frontend (React/Angular/Next.js), Kafka messaging, and formal enterprise CI/CD tooling (GitLab, SonarQube, Cucumber BDD). Your React/Next.js and REST API experience transfers directly; the delta is the JVM backend stack and a few enterprise-specific tools. The JD's "highly valued" GenAI/LLM/ AI-agent list is a genuine strength for you – lead with it.

MUST learn (Class A – explicitly required)

Java 21+ & Spring Boot 3.x – the JVM equivalent of your Node.js/Express work: `@RestController` + `@Service` + `@Repository` maps closely to an Express route handler + service function + DB query, just with annotations instead of manual wiring. 20%: create a Spring Boot REST controller returning JSON, understand `@Autowired` dependency injection, and Spring Data JPA (like Prisma, but Hibernate-based).

Kafka – distributed event streaming for async, decoupled service communication (producers publish to topics, consumers read them). Bridge: similar in spirit to the async job pattern you use (Inngest in Vibe), but for cross-service messaging. 20%: know producer/consumer/topic/partition vocabulary.

OpenAPI / TypeSpec – formal API-contract spec formats (OpenAPI/Swagger describes a REST API in YAML/JSON; TypeSpec compiles to OpenAPI). You already design REST APIs – this formalizes the contract. 20%: read/write a basic OpenAPI YAML spec for one endpoint.

JUnit 5 & Cucumber BDD – Java's unit-testing framework and a Gherkin-based (Given/When/Then) behavior-driven testing framework. A genuine gap (no tests in your repos today) but a natural extension of your debugging instincts. 20%: write one JUnit test and one Cucumber feature file.

GitLab & SonarQube – GitLab = Git hosting + CI/CD (like GitHub + Actions combined); SonarQube = static code-quality/security scanning in CI. 20%: know what a SonarQube "quality gate" is (a pass/ fail threshold blocking a merge).

GOOD to know / already strong (genuine differentiator)

AI-assisted developer tools (GitHub Copilot, Claude Code) – you are a genuine daily user of Claude Code and Cursor AI; this is explicitly called out in the JD as a required skill. Lead with it.

GenAI, LLMs, AI Agents, prompt engineering, spec-driven development with AI agents – all genuine strengths from Echo (RAG, vector search, agentic AI support workflow) and Vibe (agentic coding loop, prompt engineering with structured system prompts). Frame your project work explicitly using this JD's exact vocabulary in interviews.

React, Next.js, REST APIs, PostgreSQL, MongoDB, Git, CI/CD – already strong, no new prep needed.

Priority if time is short

1. Spring Boot basics (biggest gap, backbone of the JD). 2. Kafka producer/consumer vocabulary (quick, high-value conceptually). 3. JUnit/Cucumber – write one example of each. 4. GitLab CI/SonarQube – just enough to discuss conceptually.